Experiment No 8

Course Outcome: CO4 **Blooms Level: L3**

Aim: Implement distance-based anomaly detection using k nearest neighbours.

Abstract: Real-world datasets often contain abnormal observations that deviate significantly from the majority of the data and may affect the accuracy of data analysis and machine learning models. Traditional data analysis techniques may fail to identify such anomalous points effectively. Therefore, there is a need for a distance-based approach that can measure the similarity between data points and identify those that behave differently from their neighbors. Apply the k-Nearest Neighbours (k-NN) algorithm using Python libraries to analyze the dataset by computing the distances between data points and their nearest neighbors. Identify observations that have significantly larger distances from their neighbors and classify them as anomalies. Visualize the detected anomalies to better understand the distribution of normal and abnormal data points in the dataset.

Theory: The k-NN algorithm is a simple and intuitive method for classification and regression tasks, but it can also be adapted for outlier detection.

Outlier detection using k-NN works as follows:

1. Data Preparation: Begin with a dataset consisting of features and corresponding target labels (if any). Ensure that the features are appropriately scaled, as k-NN is sensitive to differences in feature scales.

2.Choosing a Value for k: Decide on the value of k, which represents the number of nearest neighbors to consider when identifying outliers. A smaller value of k may be more sensitive to outliers, while a larger value of k may be more robust but might miss local outliers.

3.Calculating Distances: For each data point in the dataset, calculate its distance to the k nearest neighbors. The distance metric used (e.g., Euclidean distance, Manhattan distance) depends on the nature of the data and the problem domain.

4.Identifying Outliers: Once distances to the k nearest neighbors are calculated for each data point, calculate a measure of 'outlierness' for each point. This measure could be based on various factors such as the distance to the kth nearest neighbor, the average distance to the k nearest neighbors, or the density of points in the neighborhood.

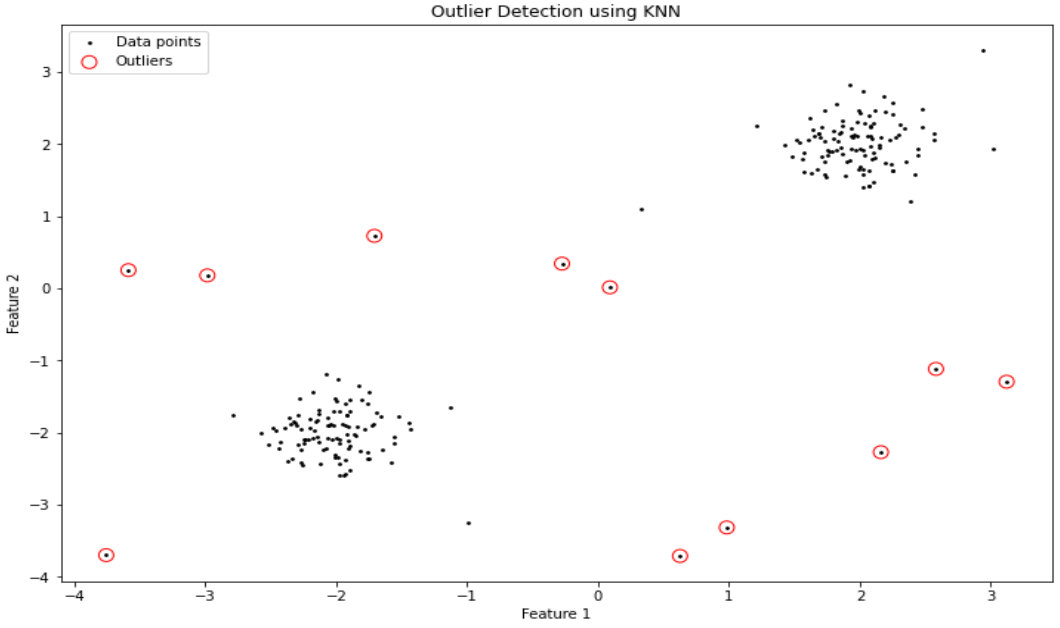
5.Setting a Threshold: Determine a threshold value that defines what constitutes an outlier. This threshold can be set manually based on domain knowledge or statistical properties of the dataset, or it can be determined empirically through techniques such as cross-validation.

6.Labeling Outliers: Finally, classify data points as outliers or non-outliers based on whether their 'outlierness' measure exceeds the chosen threshold.

Program:

In [1]:
```
from sklearn.neighbors import NearestNeighbors
import numpy as np
import matplotlib.pyplot as plt

# Generating sample data
np.random.seed(42)
X = 0.3 * np.random.randn(100, 2)
X_outliers = np.random.uniform(low=-4, high=4, size=(20, 2))
X = np.r_[X + 2, X - 2, X_outliers]

# Fit the nearest neighbors model
k = 10  # Number of neighbors
knn = NearestNeighbors(n_neighbors=k)
knn.fit(X)

# Calculate distances to k-th nearest neighbors for each point
distances, _ = knn.kneighbors(X)
kth_distance = distances[:, -1]

# Set threshold for outlier detection
threshold = np.percentile(kth_distance, 95)  # Adjust the percentile based on your data

# Detect outliers
outlier_indices = np.where(kth_distance > threshold)[0]
outliers = X[outlier_indices]

# Plot the results
plt.figure(figsize=(12, 8))

plt.scatter(X[:, 0], X[:, 1], color='k', s=3, label='Data points')
plt.scatter(outliers[:, 0], outliers[:, 1], marker='o', edgecolors='r', facecolors='none', s=100, label='Outliers')
```

```
plt.title('Outlier Detection using KNN')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



Conclusion: KNN outlier detection relies on the premise that outliers have relatively few neighbors within a certain distance threshold. The algorithm involves fitting a KNN model to the data and calculating the distances to the k-nearest neighbors for each point. By setting a threshold based on these distances, points that fall beyond this threshold are considered outliers

Exercise 1:

Abstract: The Wine dataset contains chemical analysis results of wines derived from three different cultivars, with multiple features representing chemical properties such as alcohol content, ash, and flavonoids. In such datasets, some observations may significantly differ from the general pattern of the data. Apply the k-Nearest Neighbours (k-NN) algorithm to analyze the dataset by computing the distances between data points and their nearest neighbors. Identify observations that show unusually large distances compared to neighboring points and classify them as anomalies. Visualize the detected anomalies to understand the distribution of normal and abnormal observations in the dataset

Dataset:

UCI Repository link: https://archive.ics.uci.edu/ml/datasets/wine

.

Exercise 2:

Abstract: The Breast Cancer Wisconsin dataset contains features computed from digitized images of breast mass samples, describing characteristics such as radius, texture, smoothness, and compactness. Variations in these features may indicate abnormal patterns in the dataset. Apply the k-Nearest Neighbours (k-NN) algorithm to analyze the dataset by measuring the distance between data points and their nearest neighbors. Identify observations that deviate significantly from the majority of the data and classify them as anomalies. Visualize the detected anomalies to interpret the distribution of normal and abnormal data points.

Dataset:

UCI Repository link: https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic)